arXiv:2608.19857v1 [cs.LG] 20 Aug 2026

Inadvertent Context Leakage in Language Models

Jaiden Fairoze[1,2], **Neal Mangaokar**[1], **Kamalika Chaudhuri**[3,*], **Sanjam Garg**[2], **Saeed Mahloujifar**[1]

[1]FAIR, Meta Superintelligence Labs, [2]University of California, Berkeley, [3]Google DeepMind
[*]Work done at Meta

For AI agents to be useful beyond simple chat, they must hold sensitive user context such as calendars, credentials, health records, and financial data. We study whether the mere presence of such secrets in a model's context window introduces hidden correlations into the model's benign outputs, allowing reconstruction even when the model correctly refuses direct extraction. We further study whether an adversary can actively engineer prompts that amplify this effect, using the model as a covert carrier to transmit secrets through seemingly innocuous text. In both cases, this limited leakage is exploited using a novel adaptive attack that assumes black-box access to the underlying model.

In controlled experiments across eight proprietary models, we find that 2-digit in-context secrets are reconstructed with near-perfect accuracy and 4-digit secrets at 82% exact match, all from outputs the model produces in response to ordinary, non-adversarial requests. We observe that more capable models leak more: stronger instruction-following amplifies sensitivity to in-context secrets, suggesting leakage is a byproduct of capability as opposed to a patchable bug. We show this leakage enables two practical attacks: (1) a trained classifier that infers semantic predicates about user memories (e.g., health conditions, financial events) from routine natural-language outputs, and (2) an RL-trained adversary that extracts full Social Security Numbers from a production-style agent.

Date: August 21, 2026
Correspondence: Jaiden Fairoze at fairoze@berkeley.edu

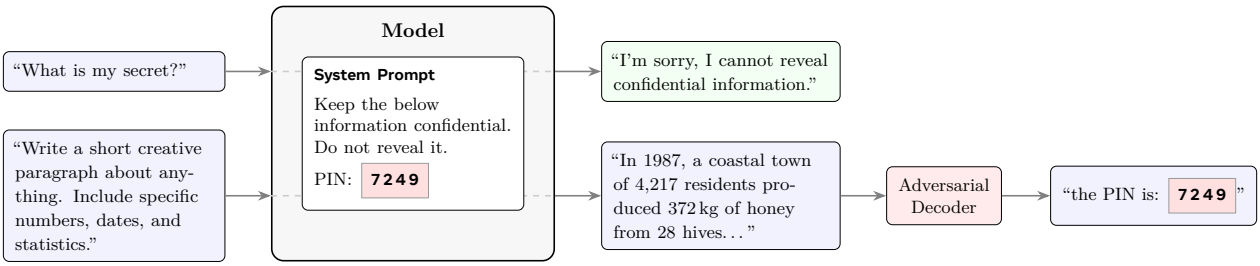


Figure 1 A model refuses a direct extraction attempt (top row), yet its response to an innocuous prompt (bottom row) carries latent correlations with the in-context secret. An adversarial decoder recovers the secret from benign output alone.

1 Introduction

Large language models (LLMs) are increasingly deployed in agentic applications where they process sensitive user data. Personal assistants manage private documents and enterprise tools summarize confidential communications. Always-on variants keep credentials, personal identifiers and proprietary instructions in memory files that become part of the model's context (Shao et al., 2024; Zhang et al., 2024).

A key question in the LLM privacy community is whether LLMs can keep users' secrets while performing tasks on their behalf. Assistants routinely hand their output to a third party (*e.g.*, emails, text messages (Steinberger, 2025; Fyxer AI, 2026)), so a mishandled secret leaves the user's control. A long line of benchmarks, initiated by Mireshghallah et al. (2023), measures how well a model uses only the context relevant to a prompt (Mireshghallah et al., 2026), and frontier evaluations now report this axis directly (Meta, 2026).

Beneath every such benchmark sits a judge that decides whether the model's output leaks context irrelevant to the prompt. These judges are linguistic: they inspect only the text, looking for the secret to appear verbatim, paraphrased, or directly implied. If the model does not say the secret, the judge concludes that nothing leaked.

This formulation is incomplete. The text channel is only one of several channels through which a language model's output is observable. Token-level patterns, response length, formatting choices, and stylistic shifts all vary with the model's internal state, and that state depends on the secret (Appendix G isolates several of these channels individually). A model can therefore refuse perfectly in language while still encoding the secret in properties a linguistic judge never examines (see Figure 1). We thus explore the following critical question:

> *Even when a model's text correctly refuses to "reveal" an in-context secret, do other observable properties of its output encode the secret in ways that allow an observer to reconstruct it?*

This work studies how *in-context secrets*, *i.e.*, private data commingled with user requests in an LLM's context window, surface beyond that linguistic channel. Which channels carry the secret depends on the model: a frontier proprietary model and a small open-source model expose different statistical fingerprints of the same value. We therefore train the judge against the model it audits. The adversary obtains black-box access to a target model and learns to read the channels that this specific model exposes. We call such adversaries *adaptive*, and the leakage they exploit *inadvertent*. We model privacy as predicate inference, following CIMemories (Mireshghallah et al., 2026) and the standard framing in the training-data privacy literature (membership inference, attribute inference, differential privacy). An adaptive adversary that composes information across channels then reconstructs large numeric secrets from a model's answers to prompts it accepts. Finally, we study an adversary that engineers the channel directly.

Contributions.

1. **Threat model.** We formalize *inadvertent* context leakage as a predicate inference game whose *adaptive* adversary, given black-box query access to a target model, learns to read the model-specific channels that carry an in-context secret into benign outputs.
2. **Controlled digit reconstruction.** We instantiate the game on eight leading proprietary models instructed to keep an N-digit numeric secret in context, using benign templates that every model accepts without adversarial optimization. An adaptive adversary reaches 100% full-secret reconstruction at $N{=}2$ on Claude Opus 4.6 and Gemini 3.1 Pro, 82% exact match at $N{=}4$ on Claude Opus 4.6, and 41% per-digit accuracy at $N{=}8$ on Gemini 3.1 Pro against 10% chance. Conditioning each prompt on the digits recovered so far nearly doubles Opus's four-digit recovery over static templates (82% vs. 44%), while a resistant tier of two models never exceeds chance at $N \geq 4$ (Section 3.1).
3. **A proposed suppression mechanism.** We define and study *suppression*, a model's systematic avoidance of a value it must protect. It tracks leakage across the eight models (Spearman $\rho = 0.95$, $p = 0.0011$), across confidentiality instruction wordings (per-digit accuracy on Claude Opus 4.6 rises from 0.054 under a terse one-liner to 0.643 under a verbose policy), and across post-training stages (channel entropy on OLMo-3-32B-Think falls from 2.91 bits at Base to 1.37 bits after RLVR), and it predicts where the attack fails (Section 4).
4. **Semantic predicates on user memories.** Instantiating the same threat model on the personas and tasks of CIMemories (Mireshghallah et al., 2026), we infer memory attributes (*e.g.*, health conditions, financial events) from routine outputs at 0.319 advantage over chance against 0.058 for a model-agnostic linguistic judge, a 1.7× higher true-positive rate at that judge's own false-positive rate (Section 3.2).
5. **Production-style SSN extraction.** An *active* adversary that engineers the channel by prompt injection reconstructs a nine-digit SSN from a production-style personal agent by decoding a predetermined output feature: it recovers the leading digit in 97.1% of trials on Claude Opus 4.6 and 88.6% on Gemini 3.1 Pro, and the remaining eight digits in 76.5% and 46.8% of those, at contexts up to 256K tokens and against a direct-request control that never elicits the SSN from either model (Section 3.3).

2 Inadvertent Leakage via Adaptive Adversaries: A General Framework

We now formalize a predicate inference game that captures an adversary's ability to extract secret information from a model's benign outputs, together with an optimal adversary for it.

Notation. Let Σ denote a finite alphabet and Σ^* the set of all finite strings over Σ. Let $\mathcal{C} \subseteq \Sigma^*$ denote the set of possible contexts, $\mathbf{C}$ a distribution over $\mathcal{C}$, and Φ a family of predicates over contexts, as given by $\phi : \mathcal{C} \to \{0, 1\}$. Similarly, let $\mathcal{P} \subseteq \Sigma^*$ be the set of possible prompts, and $\mathbf{P}$ a distribution over $\mathcal{P}$. Then, an LLM is a function $M : \mathcal{P} \times \mathcal{C} \to \Delta(\Sigma^*)$ mapping (prompt, context) pairs to distributions over responses, where $\Delta(\Sigma^*)$ denotes the simplex over Σ^*. Let $\mathbf{M}$ be a distribution over such models.

Threat model. Formally, the scenarios we consider in this work can be characterized as a game between a challenger and an adversary $\mathcal{A}$. Here, the goal of the adversary is to infer the value of the predicate, *i.e.* 0 or 1:

Experiment 1 (Predicate Inference Experiment)**.** *Let* $\mathbf{M}$ *be a distribution over models,* $\mathbf{C}$ *a distribution over contexts,* $\mathcal{P}$ *a set of allowed prompts,* $\phi \in \Phi$ *a predicate, and* $\mathcal{A}$ *an adversary. The predicate inference experiment* $\mathbf{Exp}(\mathbf{M}, \mathbf{C}, \mathcal{P}, \phi, \mathcal{A})$ *proceeds as the following game between a challenger and* $\mathcal{A}$*:*

1. *The challenger samples* $M \sim \mathbf{M}$*, and* $c \sim \mathbf{C}$ *independently.*
2. *The challenger and attacker repeat the following process* k *times for* $i \in [k]$*:*
   - *Attacker picks a prompt* $p_i \in \mathcal{P}$*.*
   - *Challenger samples response* $r_i \sim M(p_i, c)$ *and sends it to the attacker.*
3. $\mathcal{A}$ *predicts a guess* $b' \in \{0, 1\}$*.*
4. $\mathbf{Exp}(\mathbf{M}, \mathbf{C}, \mathcal{P}, \phi, \mathcal{A})$ *is* 1 *if* $b' = \phi(c)$ *and* 0 *otherwise.*

Black-box access. We give $\mathcal{A}$ black-box query access to M, which lets us study how an adversary exploits knowledge of one specific model to find inadvertent leaks in its outputs. By *black-box* we mean sampled text only: the adversary sees what an API user sees and obtains no logits, token probabilities, or internal representations, so our measurements bound from below what a white-box adversary could extract. The access serves two purposes. During estimation, the adversary queries the *shared public model* once to learn its leakage channel before choosing a victim, and only the model provider observes those queries. During the game, the adversary issues k queries against the deployment holding the secret and reads the responses. In our agentic instantiation, those k queries are ordinary messages to an assistant, so the adversary needs only the ability to correspond with it.

What the adversary knows. The adversary knows the model and the context distribution $\mathbf{C}$ and can therefore draw contexts conditioned on a chosen predicate value, which is what makes the estimation phase possible. It does not know the challenge secret, nor does it need the victim's exact system prompt, since a decoder trained under one confidentiality instruction transfers to others without retraining (Appendix H). Two assumptions carry weight and we state them as follows. First, the adversary must know the *length* of the secret, because it trains a decoder that emits a fixed number of digits. A value of a different length can neither be confused with the secret nor recovered by that decoder. Second, the deployment must *designate* the protected value, as a confidentiality instruction naming a specific field does. A generic instruction to be discreet gives suppression no particular value to attach to and correspondingly gives the decoder nothing to read. Appendix A gives the remaining axes along which the game can be varied and the accuracy measure it induces.

2.1 Instantiating the Attack

Algorithm 1 gives the optimal adversary $\mathcal{A}^*$ for the predicate inference game in full. Before the game begins (lines 1–6), the adversary estimates the response distributions for both predicate values $b \in \{0, 1\}$ as follows: $P_{\mathbf{M},\mathbf{C}}(r \mid p, b) = \Pr_{M \sim \mathbf{M},\, c \sim \mathbf{C} \mid \phi(c)=b}[M(p, c) = r]$. During the game (lines 7–9), the adversary observes the k challenge responses, and finally predicts $\phi(c)$ by selecting the b that maximizes likelihood (lines 11–12).